AmberMD Project Report

Version 1.0 - 05/08/2026

Team Members:

Luciano Aldana
Gavin Chan
Braedon Edison
Leonardo Pahtle Quehol
Christian Perez
Benjamin Saucedo
Alejandro Urbano
Luis Vicente Cruz
Isaiah Villalobos
Ira Vizcarra
Astrid Zepeta

Faculty Advisor / Liaison:

Negin Forouzesh

Graduate Student Assistant:

Nikhil Dhiman

Table of Contents

Introduction

Background

Amber Molecular Dynamics (Amber MD) is a widely used software suite for simulating biomolecules, including proteins, nucleic acids, and other complex molecular systems. It is extensively utilized in fields such as computational chemistry, drug discovery, and biophysics due to its accurate force fields and comprehensive tools for system preparation, simulation, and analysis. Over time, the Amber community has generated a vast amount of knowledge through mailing list archives, tutorials, and documentation, which serve as essential resources for both new and experienced users.

However, these resources present a significant challenge. The Amber mailing list archives span decades of discussions, containing valuable problem-solving insights, technical explanations, and expert recommendations. While this information is highly useful, it is also large, unstructured, and difficult to navigate efficiently. Users are often required to manually search through extensive documentation, frequently encountering irrelevant results or struggling to locate precise answers to specific questions. This process can be time-consuming and discouraging, particularly for beginners who are unfamiliar with Amber's workflows and terminology.

Existing approaches rely heavily on keyword-based searches or manual browsing, which do not effectively capture the semantic meaning of user queries. As a result, users may miss relevant discussions or fail to connect related concepts across different sources. This limitation highlights the need for a more intelligent system capable of understanding natural language queries and retrieving contextually relevant information.

To address this challenge, the Amber AI Assistant project was developed as a Retrieval-Augmented Generation (RAG) system designed to transform the existing Amber knowledge base into an interactive, AI-powered chatbot. By combining modern natural language processing techniques with structured data pipelines, the system enables users to query Amber-related information more efficiently and receive clear, context-aware responses grounded in authoritative sources.

Member Contribution

The development of the Amber AI Assistant project was divided into specialized teams, each responsible for a core component of the system. This division of work allowed for parallel development and ensured that each subsystem was implemented with focused attention to detail.

The Database Team (Braedon Edison, Luis Vicente Cruz) was responsible for designing and implementing the vector database used for semantic retrieval. This included configuring ChromaDB as the primary storage system, organizing all Amber data into a unified collection, and ensuring that each document stored contained its corresponding text, metadata, and embedding. The team also ensured that the database structure supported efficient similarity search and persistent storage using a DuckDB and Parquet backend.

The Data Scraping Team (Luciano Aldana, Gavin Chan, Alejandro Urbano) developed the data ingestion pipeline that collects raw data from Amber mailing list archives and tutorial websites. Their work included building web-scraping scripts to download HTML content, parsing it into a structured JSON format, and merging individual messages into thread-level documents. They also implemented mechanisms, such as UUID-based identifiers, to maintain consistency and prevent duplication across datasets.

The RAG Team (Leonardo Pahtle Quechol, Christian Perez, Benjamin Saucedo, Isaiah Villalobos, Ira Vizcarra) focused on building the core Retrieval-Augmented Generation pipeline. This included generating embeddings using the MiniLM-L6-v2 model, integrating a local large language model through Ollama with Llama3, and developing the query and retrieval system. The team also performed fine-tuning and evaluation by adjusting similarity thresholds, testing different prompt strategies, and improving response accuracy while minimizing hallucinations.

The Jupyter Notebook Team (Luciano Aldana, Gavin Chan, Braedon Edison, Alejandro Urbano, Luis Vicente Cruz) was responsible for creating interactive tutorial environments to enhance accessibility and usability. They developed reproducible Jupyter Notebook workflows that guide users through Amber tutorials, ensuring consistent execution environments through containerization and dependency management. This component supports learning and practical application alongside the chatbot system.

Together, these teams contributed to a modular and scalable system that integrates data ingestion, storage, retrieval, and user interaction into a cohesive AI-powered assistant.

Key Design Principles

The design of the Amber AI Assistant was guided by a set of core principles intended to ensure reliability, scalability, and ease of use. These principles influenced both architectural decisions and implementation strategies throughout the project.

One of the primary principles followed was the KISS (Keep It Simple, Stupid) principle. The system was intentionally designed to minimize unnecessary complexity by using straightforward, well-supported technologies and a modular pipeline structure. This approach allowed each subsystem, such as data ingestion, embedding generation, and retrieval, to remain independent and easier to debug, maintain, and extend over time.

Another key principle was reliability over optimization. Instead of prioritizing maximum performance or advanced optimizations, the system was designed to consistently deliver accurate, stable results. This is particularly important in a knowledge retrieval system, where incorrect or inconsistent outputs can significantly reduce user trust. As a result, emphasis was placed on validation, structured data processing, and controlled retrieval mechanisms to ensure dependable responses.

The system also follows an offline-first and self-contained design philosophy. All data, embeddings, and models are stored and executed locally, allowing the system to function without reliance on external cloud services. This decision improves reproducibility, reduces dependency on network availability, and aligns with environments where sensitive or large-scale scientific data must be processed locally.

A modular pipeline architecture was another essential design principle. The system is divided into clearly defined stages, including data scraping, parsing, validation, embedding generation, storage, and retrieval. This separation of concerns allows individual components to be updated or replaced without affecting the entire system, supporting long-term scalability and maintainability.

Finally, the system was designed with a user-centered approach, focusing on providing a simple and intuitive interface similar to modern AI chat assistants. By abstracting complex backend processes and allowing users to interact through natural language queries, the system reduces the learning curve and makes Amber resources more accessible to both new and experienced users.

Design Benefits

The Amber AI Assistant provides several key benefits that improve how users interact with Amber Molecular Dynamics resources. By transforming large, unstructured datasets into an accessible and intelligent system, the project significantly enhances both efficiency and usability.

One major benefit is improved efficiency in information retrieval. Instead of manually searching through extensive mailing list archives and documentation, users can submit natural language queries and receive relevant, context-aware responses. This reduces the time required to locate useful information and allows users to focus more on problem-solving rather than searching.

Another important benefit is increased accuracy and relevance of results. By using a Retrieval-Augmented Generation (RAG) approach, the system grounds its responses in actual source documents rather than relying solely on a language model's internal knowledge. This helps reduce hallucinations and ensures that answers are supported by real Amber discussions, tutorials, and documentation.

The system also provides scalability and maintainability through its modular pipeline design. Each subsystem, including data ingestion, embedding generation, storage, and retrieval, operates independently. This makes it easier to update or expand specific components, such as adding new data sources or improving the retrieval algorithm, without requiring a complete redesign of the system.

Another key benefit is automation and continuous data updates. Through the use of workflow orchestration tools, the system can automatically scrape, process, and update new Amber data on a scheduled basis. This ensures that the knowledge base remains current and continues to grow over time without requiring constant manual intervention.

Finally, the project enhances accessibility and user experience. By presenting complex technical information through a chatbot-style interface, the system lowers the barrier to entry for new users while still supporting advanced queries from experienced users. This makes Amber resources more approachable and easier to navigate for a wider audience.

Related Technologies

Programming Languages

- **Python**: The primary language used for the system's implementation, data processing, and machine learning components.

AI, LLMs & Machine Learning

- **Llama**: The Large Language Model (LLM) is utilized within the Retrieval-Augmented Generation (RAG) pipeline to provide answers based on retrieved data.
- **Hugging Face**: The platform and community used to access the embedding models and related technical documentation.

Databases & Storage

- **Chroma (ChromaDB)**: A local vector database optimized for storing high-dimensional embeddings and performing similarity searches.

Workflow & Orchestration

- **Apache Airflow**: The tool responsible for scheduling and managing the execution of the ingestion, parsing, and embedding tasks.

Data Collection & Web Tools

- **BeautifulSoup**: A library used within the parsing and scraping subsystems to convert raw HTML into structured data.
- **Socket.io**: A library used for enabling real-time, bi-directional communication between the web client and the server.

Scientific & Development Tools

- **Amber (Amber Molecular Dynamics)**: The suite of biomolecular simulation programs that serves as the core knowledge base and target domain for the assistant.
- **Jupyter**: An interactive development environment often used for testing and prototyping data processing scripts.

System Architecture

The Amber AI Assistant is designed using a modular, pipeline-based architecture that transforms raw Amber Molecular Dynamics data into an intelligent, queryable system. The architecture consists of multiple interconnected subsystems that handle data ingestion, processing, storage, and retrieval.

At a high level, the system acts as an intermediary between external data sources and the user. Raw data is collected from Amber mailing list archives and tutorial websites, processed into structured formats, and stored in a vector database. When a user submits a query, the system retrieves relevant information using semantic search and generates a response using a language model.

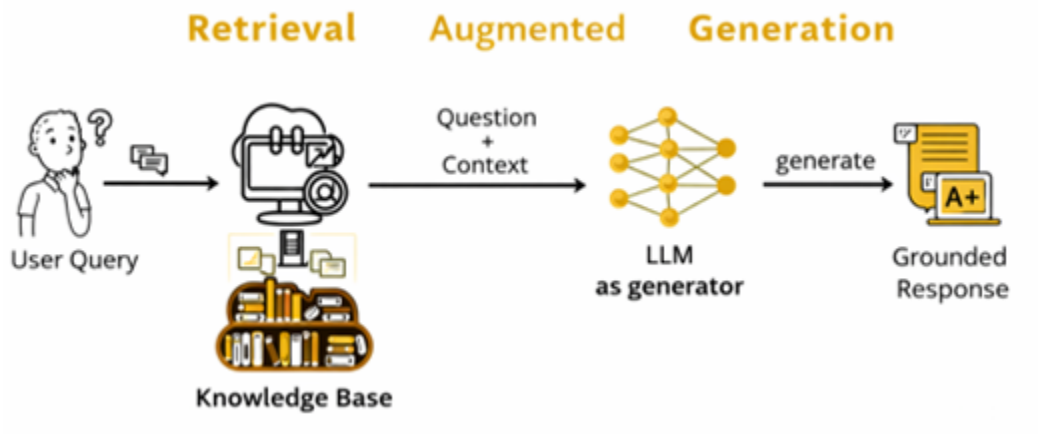

Figure 1: High-Level System Overview of the Amber AI Assistant

---

The system supports user interaction via a natural language interface, allowing users to submit queries related to Amber Molecular Dynamics. The system processes these queries and returns relevant answers based on retrieved knowledge. In addition to standard users, administrative roles are supported for monitoring system performance, managing data, and maintaining the pipeline.

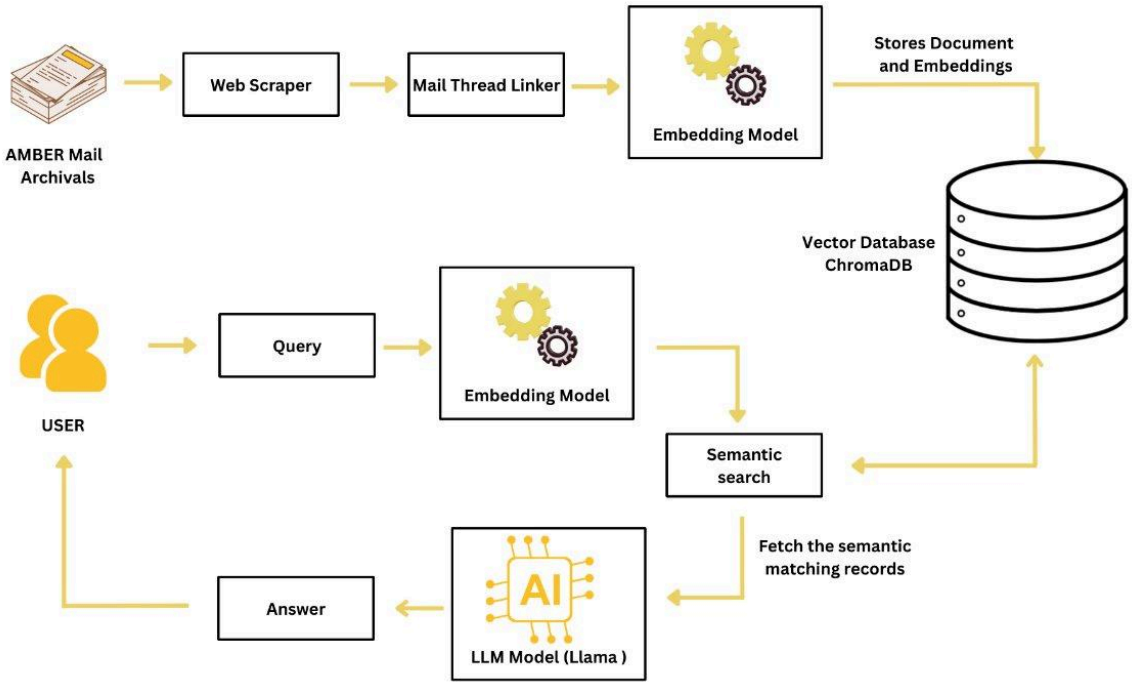

Figure 2: Use Case Diagram for User and System Interaction

The system begins with the Data Ingestion and Scraping subsystem, which collects raw HTML data from the Amber mailing list archives and tutorial pages. Custom scraping scripts download and store this data locally, ensuring that the system has access to a large and continuously growing dataset.

Next, the Parsing and Structuring subsystem converts the raw HTML data into structured JSON format. Individual messages are cleaned, processed, and merged into thread-level documents, which better represent how discussions occur in the Amber community. Each thread is assigned a unique identifier to maintain consistency and prevent duplication.

The structured data then passes through the Validation and Cleaning subsystem, which ensures that all documents meet the required schema and removes any malformed or incomplete data. This step is critical for maintaining the integrity of the system.

Once validated, the data is processed by the Embedding Generation subsystem, which converts each document into a numerical vector representation using the MiniLM-L6-v2 model. These embeddings capture the semantic meaning of the text, enabling the system to perform similarity-based searches.

The resulting embeddings and their associated metadata are stored in the Vector Storage subsystem, implemented using ChromaDB with a DuckDB and Parquet backend. This allows for efficient similarity search and persistent storage.

The Query and Retrieval subsystem handles user queries. When a user submits a query, the system generates an embedding and retrieves the most relevant documents using similarity search. These documents are then passed into a language model to generate a context-aware response.

To support automation and reliability, the system includes a Workflow Orchestration subsystem using Apache Airflow. This component schedules tasks such as scraping, parsing, embedding, and storage, while also providing monitoring and logging capabilities.

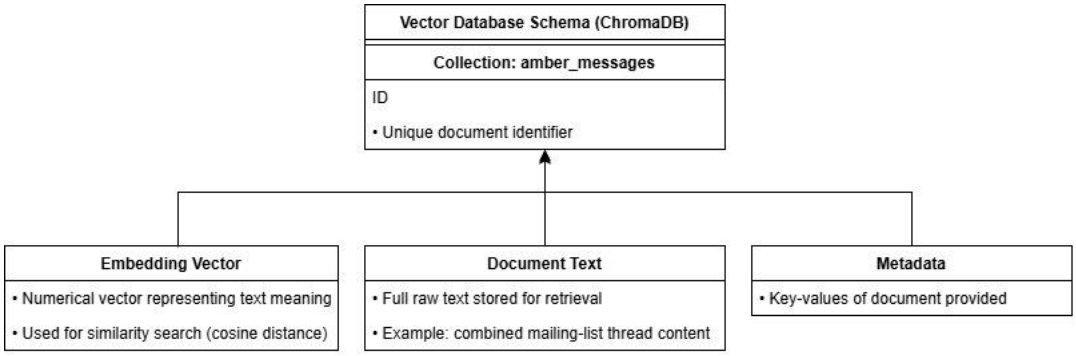

Figure 3: Data Flow of the Amber AI Assistant Pipeline

Results

The Amber AI Assistant was evaluated to determine the effectiveness of its Retrieval-Augmented Generation (RAG) pipeline in producing accurate and relevant responses to user queries. The evaluation focused on comparing the performance of the combined LLM + RAG system against standalone language models.

To measure performance, several evaluation metrics were used. Cosine similarity was used to determine how closely the generated responses matched expected answers by comparing vector embeddings. Keyword overlap measured the number of shared terms between generated responses and reference answers. Additionally, a cross-encoder model was used to compute a more precise relevance score by evaluating the similarity between queries and responses in context.

The first set of results demonstrates that the LLM + RAG pipeline consistently produces responses that are more semantically similar to the correct answers than a standalone language model. This indicates that grounding the model in retrieved documents significantly improves the quality and reliability of responses.

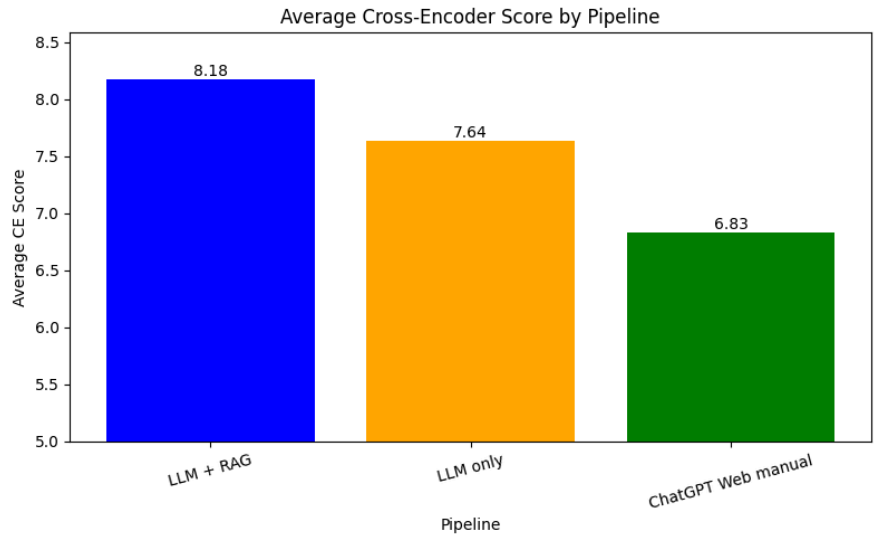

Figure 4: Cross-Encoder Similarity Comparison Between LLM and LLM + RAG

Further analysis shows that while the RAG-based system may generate slightly shorter responses, those responses are more relevant. This suggests that the system can filter out unnecessary or irrelevant information and focus on the most useful content when generating answers.

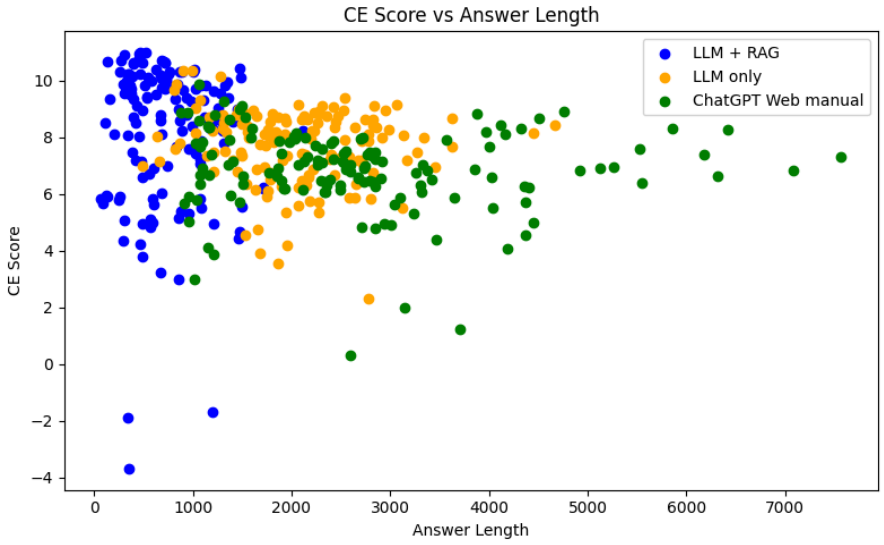

Figure 5: Relationship Between Answer Length and Similarity Score

---

Another important observation comes from the keyword overlap evaluation. Although the RAG system does not always produce the highest keyword overlap, it still achieves better overall semantic similarity. This highlights that the system is not simply matching keywords, but instead understanding the meaning of the query and retrieving contextually relevant information.

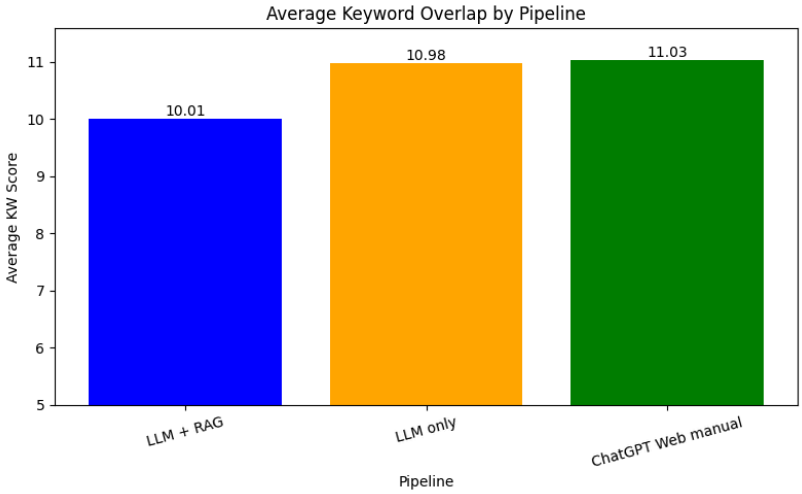

Figure 6: Keyword Overlap and Document Retrieval Performance

In addition to quantitative evaluation, qualitative results further demonstrate the system's effectiveness. The chatbot can generate step-by-step solutions and provide context-aware answers based on Amber documentation. Compared to human expert responses from the Amber mailing list, the system produces answers that are structured, informative, and grounded in relevant sources.

However, several challenges were identified during testing. The system required careful tuning of similarity thresholds to balance precision and recall in document retrieval. In some cases, retrieving too many documents introduced noise, while retrieving too few limited the quality of generated responses. Additionally, reducing hallucinations required iterative prompt refinement and evaluation of different model configurations.

Overall, the results indicate that integrating retrieval mechanisms with language models significantly improves answer quality, making the Amber AI Assistant a reliable and effective tool for navigating complex technical knowledge.

Conclusion

The Amber AI Assistant project successfully demonstrates the application of Retrieval-Augmented Generation (RAG) to improve access to complex technical knowledge within the Amber Molecular Dynamics ecosystem. By integrating data scraping, structured processing, vector embeddings, and language model generation into a unified pipeline, the system transforms large volumes of unstructured information into a searchable and user-friendly interface.

Through the development process, the project achieved its primary objective of enabling users to efficiently query Amber-related resources using natural language. The system can retrieve relevant information from mailing list archives and tutorials and generate context-aware responses grounded in real data. Evaluation results showed that the RAG-based approach significantly improves semantic accuracy and relevance compared to standalone language models, confirming the effectiveness of combining retrieval with generation.

In addition to its technical achievements, the project highlights the importance of modular design and system scalability. By organizing the system into independent subsystems such as data ingestion, embedding generation, storage, and retrieval, the architecture supports future expansion and maintainability. The use of local storage and an offline-first design further ensures the system remains reliable and reproducible across different environments.

Despite these successes, several challenges were encountered during development. Fine-tuning retrieval thresholds, balancing precision and recall, and reducing hallucinations required iterative testing and refinement. These challenges emphasize the complexity of building reliable AI systems and the importance of careful evaluation and validation.

Future work will focus on enhancing the system's capabilities and usability. Planned improvements include enabling the chatbot to ask follow-up questions to better understand user intent, integrating a ticketing system for unresolved queries, expanding the dataset with additional Amber resources, and refining the user interface. Additionally, further optimization of the retrieval and generation pipeline may improve performance and response quality.

Overall, the Amber AI Assistant provides a strong foundation for an intelligent knowledge retrieval system within the Amber community. By making technical information more accessible and easier to navigate, the project has the potential to significantly improve user experience and support continued research and development in molecular dynamics.

References

Amber Molecular Dynamics Official Website.
https://ambermd.org

Amber Mailing List Archive.
https://archive.ambermd.org

Amber Molecular Dynamics Tutorials.
https://ambermd.org/tutorials/

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.
https://www.sbert.net

ChromaDB Documentation.
https://docs.trychroma.com

DuckDB Documentation.
https://duckdb.org

Apache Airflow Documentation.
https://airflow.apache.org/docs

BeautifulSoup Documentation.
https://www.crummy.com/software/BeautifulSoup/bs4/doc

IBM Research. Retrieval-Augmented Generation Overview.
https://research.ibm.com/blog/retrieval-augmented-generation